Rapport de TP — Segmentation sémantique encodeur-décodeur sur Pascal VOC

1. Introduction et objectifs

La segmentation sémantique consiste à classer chaque pixel d'une image. Plutôt que de dire « cette image contient un chat », le modèle indique, pour chaque pixel, s'il appartient au chat, à une personne, au canapé ou au fond. Le résultat est une image de même taille que l'entrée, coloriée par classe, qui délimite précisément la forme de chaque objet.

On la distingue de trois tâches voisines :

- la classification d'image, qui donne un seul label pour toute l'image (TP1) ;
- la détection d'objets, qui encadre chaque objet par une boîte englobante ;
- la segmentation d'instances, qui sépare en plus les objets d'une même classe. Deux personnes côte à côte sont deux instances distinctes, alors qu'en segmentation sémantique elles forment une seule région `person`.

La sortie doit avoir la même résolution que l'entrée. L'architecture doit donc à la fois extraire une information sémantique globale (*quoi*) et conserver une précision spatiale fine (*où*). C'est ce qui motive les modèles encodeur-décodeur étudiés ici.

Le jeu Pascal VOC 2012 compte 21 classes : le fond et 20 objets (personnes, animaux, véhicules, objets d'intérieur). L'étiquette 255 marque les contours des objets et n'est pas prise en compte dans l'apprentissage.

Objectifs du TP :

1. construire un pipeline de données avec des augmentations adaptées à la segmentation ;
2. implémenter un U-Net dont l'encodeur est un ResNet-18 pré-entraîné (timm) ;
3. entraîner le modèle avec une Focal Loss, adaptée au fort déséquilibre des classes ;
4. le comparer à DeepLabV3 (torchvision), et rendre son encodeur interchangeable grâce à l'API timm.

Applications concrètes : la segmentation sémantique sert dès qu'il faut connaître la forme exacte d'un objet, et pas seulement sa position : conduite autonome (route, piétons), imagerie médicale (organes, lésions), robotique. La même architecture encodeur-décodeur a par exemple été utilisée dans un projet de vol autonome d'un drone DJI Tello, mené dans le cadre du cours *Contrôle de robots mobiles* : un U-Net à encodeur MobileNetV3-small pré-entraîné segmente l'anneau d'un cerceau dans le flux vidéo du drone, et la forme de cet anneau permet de calculer la position 3D du cerceau pour le traverser. Le projet et ses rapports sont disponibles sur GitHub : drone-tello-cerceaux.

2. Données et pipeline de prétraitement

2.1 Exploration du jeu de données

Élément	Valeur
Split utilisé	`image_set="train"` de VOC2012, découpé 80 % entraînement / 20 % validation
Entrée `x`	`torch.Tensor` `[3, 256, 256]`, `float32`
Cible `y`	`torch.Tensor` `[256, 256]`, `uint8` (valeurs 0–20, plus 255)
Nombre d'images	1 464 images annotées dans `train` → 1 171 entraînement / 293 validation (`int(0.8 × 1464)`)
Non utilisé	split officiel `val` : 1 449 images (utilisable comme jeu de test)

Paires image / masque. Chaque exemple de Pascal VOC associe une photo (`JPEGImages/<id>.jpg`) à son masque

(`SegmentationClass/<id>.png`). Le masque est annoté à la main et donne pour chaque pixel l'indice de sa classe : 0 pour le fond, 1 à 20 pour les objets, 255 pour les contours ambigus, qui sont ignorés. Le réseau ne voit jamais le masque. Celui-ci sert de cible pour la perte et de référence pour les métriques.

Chargement : `WrappedDataset` . Cette classe enveloppe le dataset torchvision. Elle convertit la paire (image, masque) de PIL vers numpy, puis applique les transformations albumentations :

```
transformed_data = self.transforms(image=X, mask=mask)
```

Comme l'image et le masque sont passés dans le même appel, ils subissent exactement la même transformation géométrique et restent alignés. Les augmentations sont refaites à chaque lecture, donc différentes à chaque époque. La classe permet aussi d'appliquer une pipeline différente aux plis d'entraînement et de validation.

Découpage entraînement / validation :

```
nb_train = int((1.0 - valid_ratio) * len(train_valid_dataset))
indices = list(range(len(train_valid_dataset)))
random.shuffle(indices)
train_indices = indices[:nb_train]
valid_indices = indices[nb_train:]

train_dataset = torch.utils.data.Subset(train_valid_dataset, train_indices)
valid_dataset = torch.utils.data.Subset(train_valid_dataset, valid_indices)
```

2.2 Redimensionnement et normalisation

```
resize_transforms = [
    A.SmallestMaxSize(max_size=crop_size),
    A.RandomCrop(height=crop_size, width=crop_size),
]
normalize_transforms = [
    A.Normalize(mean=mean, std=std),
    ToTensorV2(),
]
```

- `SmallestMaxSize(256)` puis `RandomCrop(256)` : toutes les images ont la même taille, ce qu'exige la constitution d'un mini-batch.
- `Normalize(mean, std)` : la moyenne et l'écart-type sont calculés uniquement sur le pli d'entraînement, pour éviter toute fuite d'information.

```
mean, std = 0.0, 1.0
if normalize:
    mean, std = compute_mean_std(train_dataset, crop_size)
```

2.3 Augmentations de données

Les transformations géométriques s'appliquent à l'image et au masque. Les transformations photométriques ne modifient que l'image. Les zones vides créées par une transformation doivent être remplies avec 255 dans le masque. Ce comportement vient de l'appel commun `transforms(image=X, mask=mask)` dans `WrappedDataset` .

Une augmentation est crédible si l'image produite pourrait exister dans VOC et si le masque reste exact pixel par pixel.

```
HorizontalFlip = [A.HorizontalFlip(p=0.5)]
GaussNoise = [A.GaussNoise(p=0.2)]
GaussianBlur = [A.GaussianBlur(blur_limit=(3, 5), p=0.2)]
ColorJitter = [
    A.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.05, p=0.5)
]

augmentation_transforms = HorizontalFlip + ColorJitter + GaussianBlur + GaussNoise

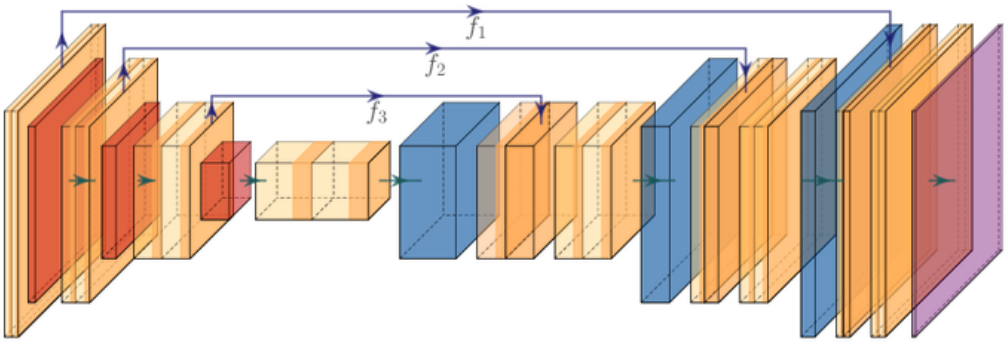
resize_transforms = [
    A.SmallestMaxSize(max_size=crop_size),
    A.RandomScale(scale_limit=0.25, p=0.5),
    A.PadIfNeeded(
        min_height=crop_size,
        min_width=crop_size,
        border_mode=0,
        fill=0,
        fill_mask=255,
    ),
    A.RandomCrop(height=crop_size, width=crop_size),
]

data_transforms = A.Compose(
    resize_transforms + augmentation_transforms + conversion_transforms
)
```

Augmentation	Paramètres	Type	Justification
`HorizontalFlip`	p = 0.5	géométrique	Retourne l'image en miroir : le réseau devient indépendant de l'orientation gauche/droite et se base sur les caractéristiques des objets.
`RandomScale` + `PadIfNeeded`	±25 %, `fill_mask=255`	géométrique	Change la taille des objets pour éviter que le réseau se base sur une taille fixe.
`ColorJitter`	0.2 / 0.2 / 0.2, hue 0.05, p = 0.5	photométrique	Altère les pixels de l'image en jouant sur la luminosité, le contraste et la saturation.
`GaussianBlur`	noyau 3–5, p = 0.2	photométrique	Adoucit l'image en mélangeant la couleur de chaque pixel avec ses voisins, sans modifier le masque.
`GaussNoise`	p = 0.2, écart-type par défaut de 20 à 44 % de l'intensité max	photométrique	Ajoute du grain sur l'image pour pousser le réseau à retrouver la forme des objets.

Écartées volontairement : `VerticalFlip` , rotations fortes, teinte forte, déformations élastiques, car elles produisent des scènes absentes de VOC (personnes à l'envers, couleurs irréalistes, objets déformés).

3. Architecture du modèle



Encodeur/décoder de type U-Net

L'encodeur, à gauche, réduit progressivement la résolution spatiale tout en enrichissant les caractéristiques extraites. Le décodeur, à droite, remonte jusqu'à la résolution d'origine par suréchantillonnages successifs (blocs bleus). Les flèches f_1, f_2 et f_3 sont les connexions de saut : elles transmettent au décodeur les cartes de caractéristiques de l'encodeur à la même résolution, ce qui lui permet de retrouver les détails fins perdus lors de la descente.

3.1 Encodeur : ResNet-18 pré-entraîné (timm)

Le `forward` du ResNet est déroulé à la main et s'arrête avant le pooling global et la couche `fc`, pour conserver des cartes de caractéristiques 2D :

```
class TimmEncoder(nn.Module):
    def __init__(self, cin):
        super().__init__()
        self.model = timm.create_model(
        model_name="resnet18", in_chans=cin, pretrained=True
        )

    def forward(self, x):
        x = self.model.conv1(x)
        x = self.model.bn1(x)
        x = self.model.act1(x)

        x = self.model.maxpool(x)

        f1 = self.model.layer1(x)
        f2 = self.model.layer2(f1)
        f3 = self.model.layer3(f2)
        f4 = self.model.layer4(f3)

        return f4, [f1, f2, f3]
```

`self.model` contient le ResNet-18 complet renvoyé par timm. Écrire `self.model.layer1(x)` revient à exécuter ce seul sous-module sur `x`. Chaque `layerX` est un étage, un `nn.Sequential` de deux blocs résiduels enchaînés qui travaillent à la même résolution : seul le premier bloc des étages 2 à 4 divise la résolution par deux et double le nombre de canaux. En appelant les étages un par un plutôt que `self.model(x)`, on s'arrête avant `global_pool` et `fc`, et on récupère au passage les sorties intermédiaires f1, f2 et f3. Ces noms sont propres au ResNet de timm.

Étape	Forme `[B, C, H, W]`	Réduction
entrée	`[1, 3, 256, 256]`	—
conv1 (stride 2)	`[1, 64, 128, 128]`	/2

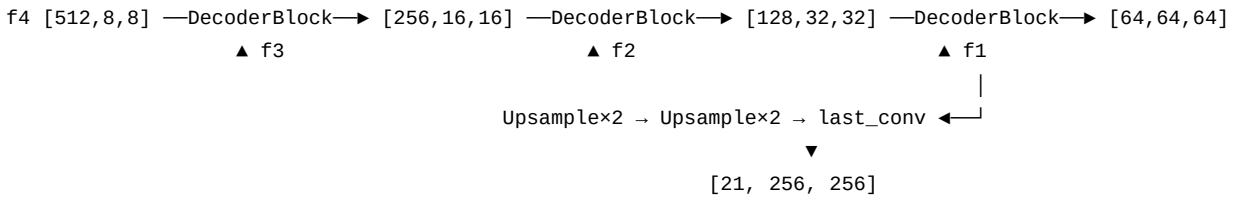
Étape	Forme `[B, C, H, W]`	Réduction
maxpool	`[1, 64, 64, 64]`	/4
f1 = layer1	`[1, 64, 64, 64]`	/4
f2 = layer2	`[1, 128, 32, 32]`	/8
f3 = layer3	`[1, 256, 16, 16]`	/16
f4 = layer4	`[1, 512, 8, 8]`	/32

La réduction de résolution vient uniquement du pas (stride) des opérations. `conv1` (noyau 7×7, pas de 2) puis le max pooling (noyau 3×3, pas de 2) divisent chacun la taille par deux, d'où le facteur /4 en sortie de `layer1`. Ensuite, la première convolution 3×3 des étages 2 à 4 a un pas de 2. Au total, cinq divisions par deux donnent 256 / 2^5 = 8. Le nombre de canaux, lui, est simplement le nombre de filtres de la convolution : il ne dépend pas de la résolution, et le doubler n'est pas ce qui la divise par deux.

Doubler les canaux à chaque réduction est donc un choix de conception. Diviser H et W par deux divise le coût par 4, et doubler C le multiplie par 4, si bien que chaque étage coûte à peu près autant. D'autre part, les premières couches détectent des motifs simples et peu nombreux (bords, textures), alors que les couches profondes encodent des parties d'objets bien plus variées, ce qui demande davantage de canaux.

Il en résulte un compromis entre le « quoi » et le « où ». f4 (512 canaux, 8×8) sait quels objets sont présents, mais localise mal leurs contours. f1 à f3 sont moins abstraits, mais spatialement précis. Les connexions de saut permettent au décodeur de combiner les deux.

3.2 Décodeur et connexions de saut



Bloc du décodeur. Il enchaîne convolutions → `Upsample ×2` → convolutions (canaux ÷ 2) → concaténation avec la connexion de saut → convolutions :
`nn.Upsample` double la résolution en recopiant chaque pixel dans un carré 2×2 (plus proche voisin) ; les convolutions qui suivent lissent ensuite ces blocs.

```
class DecoderBlock(nn.Module):
    def __init__(self, cin):
        super().__init__()
        self.conv1 = nn.Sequential(nn.Conv2d(cin,cin,kernel_size=3,stride=1, padding=1),nn.ReLU(),nn.BatchNorm2d(c
                                   nn.Conv2d(cin,cin,kernel_size=3,stride=1, padding=1),nn.ReLU(),nn.BatchNorm2d(c
        self.up_conv =nn.Sequential(nn.Upsample(scale_factor=2))
        self.conv2 = nn.Sequential(nn.Conv2d(cin,cin//2,kernel_size=3,stride=1, padding=1),nn.ReLU(),nn.BatchNorm2
                                   nn.Conv2d(cin//2,cin//2,kernel_size=3,stride=1, padding=1),nn.ReLU(),nn.BatchNo
        self.conv2bis = nn.Sequential(nn.Conv2d(cin,cin//2,kernel_size=3,stride=1, padding=1),nn.ReLU(),nn.BatchNo
                                   nn.Conv2d(cin//2,cin//2,kernel_size=3,stride=1, padding=1),nn.ReLU(),nn.BatchNo

    def forward(self, x, f_encoder):
        x = self.conv1(x)
        x=self.up_conv(x)
        x= self.conv2bis(x)
        x = torch.cat([x, f_encoder],dim=1)
        out = self.conv2 (x)
        return out
```

Assemblage du décodeur :

```
def forward(self, f4, f_encoder):
    [f1, f2, f3] = f_encoder

    x1 = self.b1(f4, f3)
    x2 = self.b2(x1, f2)
    x3 = self.b3(x2, f1)

    out = self.last_conv(self.up2(self.up1(x3)))

    return out
```

La dernière couche (`last_conv`) produit 21 cartes de scores, une par classe. La sortie a donc la forme `[B, 21, 256, 256]` : chaque pixel reçoit un vecteur de 21 scores.

À l'entraînement, un *softmax* sur la dimension des classes transforme ces scores en probabilités qui somment à 1 pour chaque pixel. La perte compare ces probabilités à la classe du masque.
On garde donc pour chaque pixel la classe de score maximal :

```
y_prob_pred = model(X_val)  # B, num_classes, H, W
y_pred = y_prob_pred.argmax(dim=1).cpu()
```

Le résultat `y_pred` , de forme `[B, 256, 256]` , est une carte d'indices de classe, au même format que le masque, et on peut les comparer directement.

3.3 Variantes

DeepLabV3 : `deeplabv3_mobilenet_v3_large` de torchvision, avec ses poids pré-entraînés. Il utilise des convolutions dilatées (ASPP) et n'a pas de décodeur U-Net.

```
class DeepLabV3(nn.Module):
    def __init__(self,input,classes):
        super().__init__()
        self. model  = segmentation.deeplabv3_mobilenet_v3_large(weights=segmentation.DeepLabV3_MobileNet_V3_Large

    def forward(self,x):
        out=self.model (x)
        return out["out"]
```

U-Net générique : timm avec `features_only=True` . Les canaux et les réductions sont lus dans `feature_info` , et les blocs du décodeur sont rangés dans une `nn.ModuleList` . On change de backbone en changeant une chaîne de caractères (`"resnet18"` , `"efficientnet_b3"` , ...).

```
class GenericTimmEncoder(nn.Module):
    def __init__(self, cin,model_name):
        super().__init__()
        self.model = timm.create_model(
            model_name=model_name, features_only=True,in_chans=cin, pretrained=True
        )
        self.channels=self.model.feature_info.channels()
        self.reductions=self.model.feature_info.reduction()

    def forward(self, x):
        Backbone_feature=self.model(x)
        return Backbone_feature[-1], Backbone_feature[:-1]
```

```
class GenericDecoder(nn.Module):
    def __init__(self, channels, first_reduction, num_classes):
        super().__init__()
        n = len(channels)
        blocks = []
        cin = channels[n - 1]
        for i in range(n - 2, -1, -1):
            cskip = channels[i]
            blocks.append(GenericDecoderBlock(cin, cskip))
            cin = cskip
        self.blocks = nn.ModuleList(blocks)

        self.up = nn.Upsample(scale_factor=first_reduction)
        self.last_conv = nn.Sequential(*conv_relu_bn(cin, num_classes))
```

L'intérêt est de ne plus dépendre de la structure interne du ResNet-18, déroulée à la main dans `TimmEncoder` : avec `features_only=True`, timm renvoie directement la liste des cartes de caractéristiques de n'importe quel encodeur, et le décodeur se construit à partir de leurs nombres de canaux. Les blocs du décodeur, en nombre variable, sont rangés dans une `nn.ModuleList` : dans une simple liste Python, PyTorch ne les enregistrerait pas comme sous-modules, et leurs poids ne seraient ni entraînés ni envoyés sur le GPU.

4. Fonction de perte et métriques

4.1 Focal Loss

Le réseau sort des scores bruts (logits), transformés en probabilités par un softmax pour chaque pixel. On note $p_{t,i}$ la probabilité que le modèle attribue à la vraie classe du pixel i. La perte implémentée dans `losses.py` est :

$$\mathrm{FL} = \frac{1}{N} \sum_{i=1}^{N} -(1 - p_{t,i} + \epsilon)^{\gamma} \log(p_{t,i}), \qquad \gamma = 2, \quad \epsilon = 10^{-10}$$

où N est le nombre de pixels dont l'étiquette n'est pas 255. Les pixels déjà bien classés ($p_{t,i}$ proche de 1), en majorité du fond, pèsent très peu, et l'apprentissage se concentre sur les pixels difficiles. Avec $\gamma = 0$, on retrouve l'entropie croisée $-\log(p_{t,i})$.

4.2 Métriques

L'accuracy (proportion de pixels bien classés) est trompeuse ici, car un modèle qui prédit « fond » partout obtient déjà un score élevé. La F1 macro, moyenne non pondérée des F1 par classe, combine pour chaque classe la précision et le rappel, et donne le même poids à toutes les classes.

Pour une classe k, on compte sur l'ensemble des pixels évalués :

- TP_k (vrais positifs) : pixels de classe k prédits k ;
- FP_k (faux positifs) : pixels prédits k qui appartiennent à une autre classe ;
- FN_k (faux négatifs) : pixels de classe k prédits dans une autre classe.

Précision et rappel de la classe k :

$$P_k = \frac{TP_k}{TP_k + FP_k} \qquad R_k = \frac{TP_k}{TP_k + FN_k}$$

Score F1 de la classe k, moyenne harmonique de la précision et du rappel :

$$F1_k = \frac{2\,P_k\,R_k}{P_k + R_k} = \frac{2\,TP_k}{2\,TP_k + FP_k + FN_k}$$

Avec $K = 21$ classes et N pixels évalués :

$$\text{Accuracy} = \frac{1}{N}\sum_{k=1}^{K} TP_k \qquad \text{F1 macro} = \frac{1}{K}\sum_{k=1}^{K} F1_k$$

5. Protocole expérimental

Hyperparamètre	Valeur par défaut
Taille de batch	16
Époques	50
Optimiseur	Adam, lr = 1e-3
Scheduler	`ReduceLROnPlateau` (patience 10, facteur 0.1 par défaut)
Taille de crop	256
Perte	Focal Loss, γ = 2
Sauvegarde du modèle	meilleure perte de validation

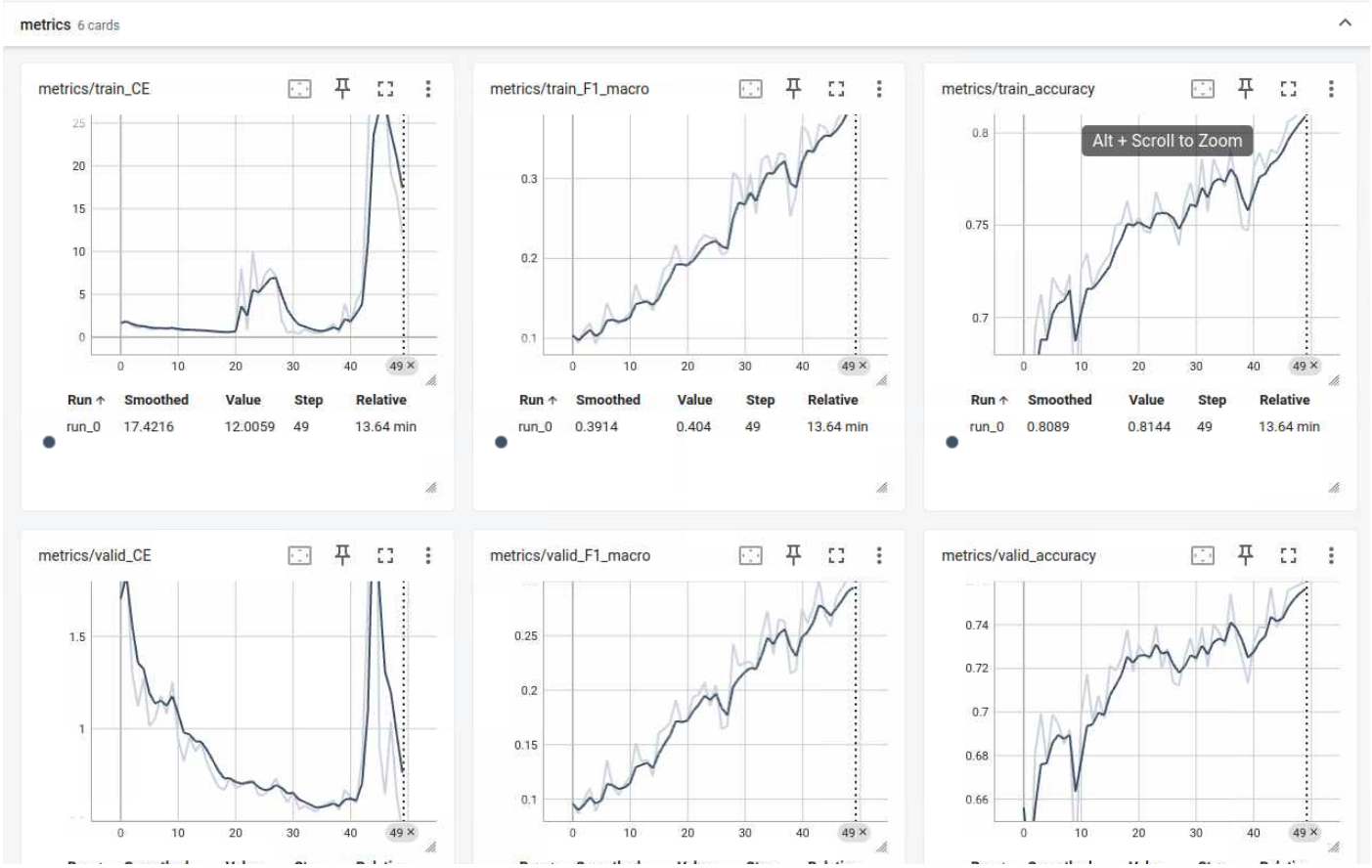
```
loss = wrap_loss(losses.FocalLoss(ignore_index=255))
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer)
```

6. Résultats et analyse

La démarche suit quatre étapes : on entraîne d'abord le U-Net ResNet-18 avec les augmentations d'origine du code (baseline), puis on mesure l'apport de nos augmentations sur ce même modèle, on étudie ensuite l'effet du taux d'apprentissage, et on compare enfin la meilleure configuration du U-Net à DeepLabV3. Tous les runs durent 50 époques avec les mêmes hyperparamètres, sauf le taux d'apprentissage au § 6.3. Les runs réalisés après la modification du code utilisent la graine 42 et sont directement comparables ; les deux premiers runs (`run_0` , `run_1`) n'ont pas de graine fixée.

6.1 Baseline : U-Net ResNet-18 avec les augmentations d'origine

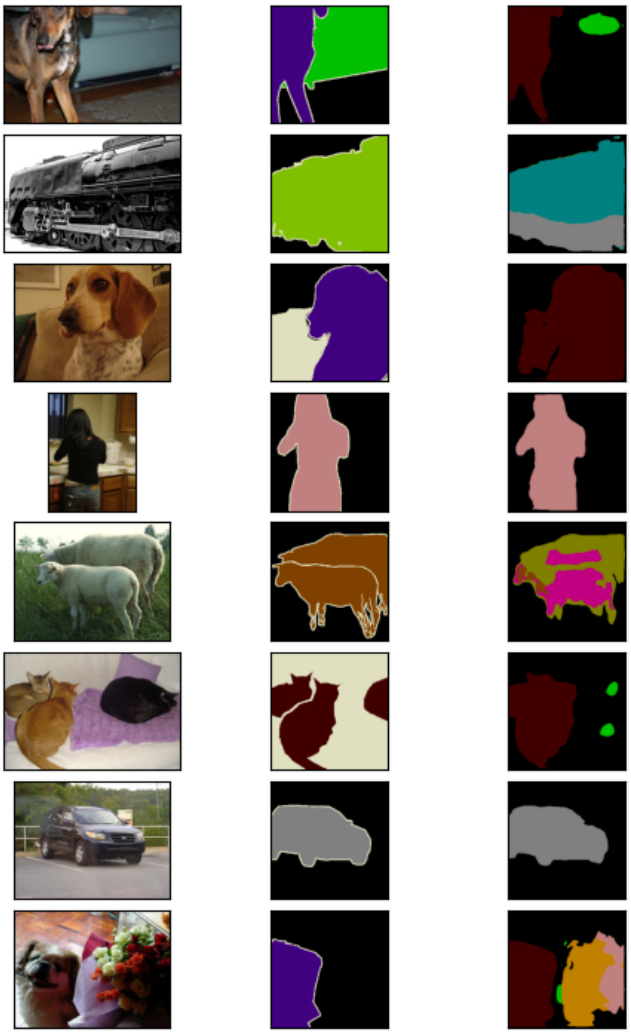
Augmentations d'origine du code : `HorizontalFlip` et `ShiftScaleRotate` (zoom de ±15 %, rotation jusqu'à 45°).



Cette figure regroupe les six courbes enregistrées dans TensorBoard pendant l'entraînement. La ligne du haut correspond au pli d'entraînement et celle du bas au pli de validation ; les colonnes donnent, de gauche à droite, la perte, le F1 macro et l'accuracy. L'axe horizontal indique le numéro d'époque. La courbe foncée est lissée par TensorBoard (lissage de 0,6), la courbe claire donne les valeurs brutes de chaque époque. On attend une perte qui diminue et des scores qui augmentent ; l'écart entre les deux lignes renseigne sur le sur-apprentissage.

Run	Paramètres	Époques	F1 train	F1 valid (meilleure)	Accuracy valid	Perte valid (min)
U-Net ResNet-18, augmentations d'origine	22,7 M	50	0,404	0,300 (0,303 à l'époque 48)	0,760	0,439 (époque 49)

Valeurs de la dernière époque, lues dans les logs TensorBoard du run.



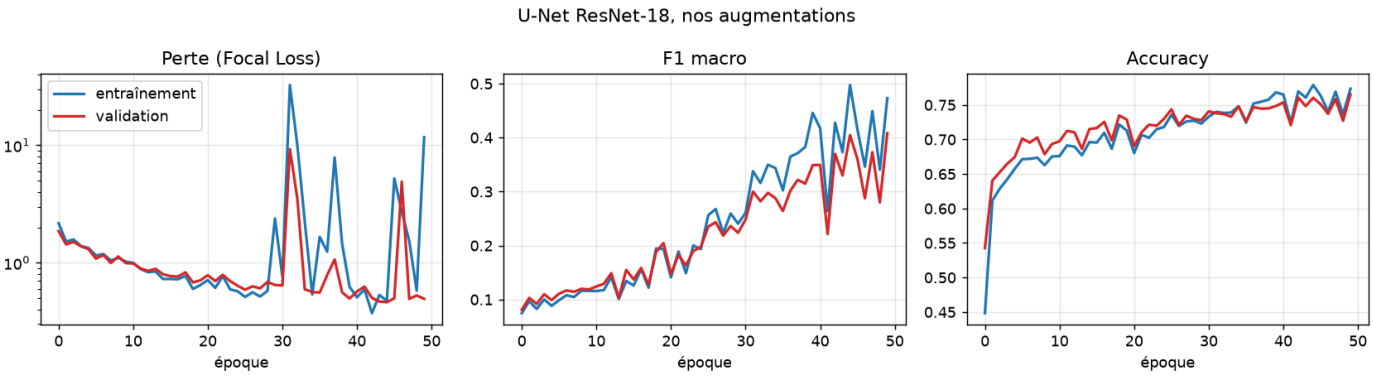
Chaque ligne montre une image de validation, son masque annoté et la prédiction du modèle, avec une couleur par classe. L'image de gauche est l'image d'origine, non recadrée, alors que le masque et la prédiction correspondent au crop de 256×256.

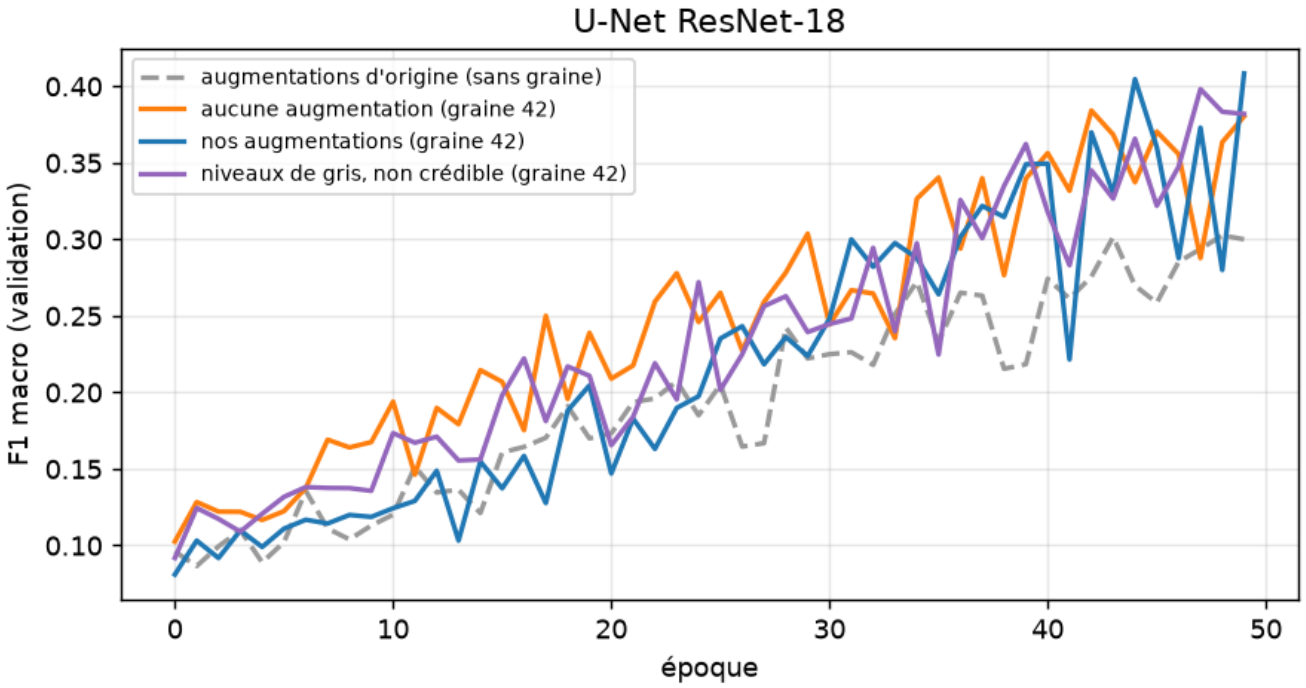
Ce run a été réalisé avant la modification du code et n'utilise pas de graine fixe : son découpage train / valid diffère de celui des runs suivants.

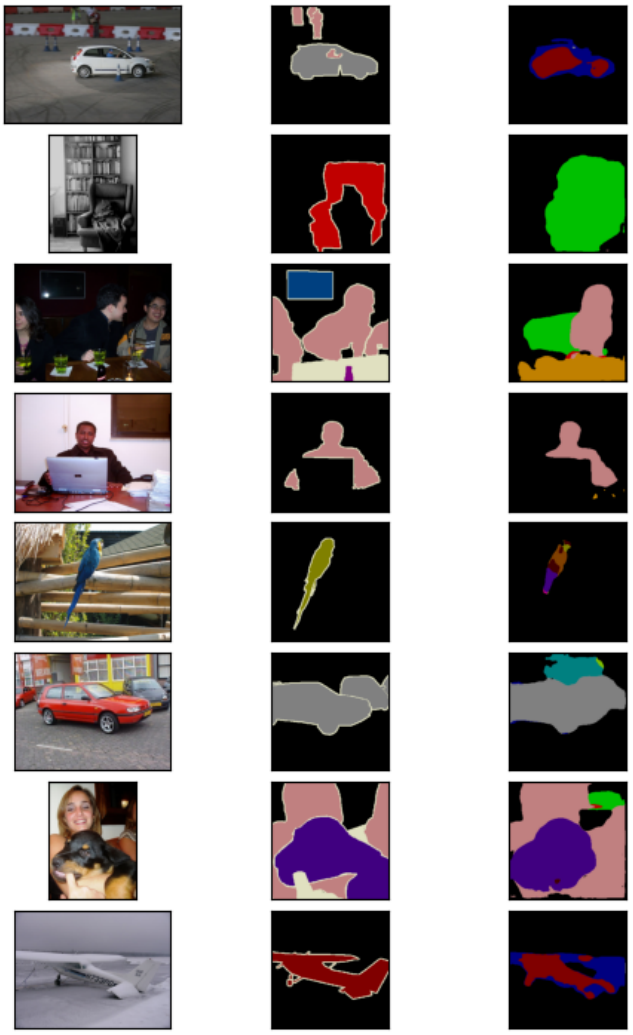
Le résultat est moyen : F1 de validation de 0,30. Le F1 par classe (sortie console) montre que plusieurs classes ne sont jamais segmentées (vélo, bateau, bouteille, chaise, plante en pot), ce qui fait chuter la moyenne macro.
Les prédictions montrent que le modèle localise assez bien les objets (formes de la personne, de la voiture), mais confond les classes proches : chiens prédits comme chats, moutons prédits comme cheval ou vache, locomotive prédite comme bus et voiture.
Les contours restent grossiers et les détails fins (rayons de la moto) disparaissent.
Le F1 et l'accuracy progressent de façon régulière pendant les 50 époques, sans plateau : le modèle n'a pas convergé.
L'écart entre F1 (0,30) et accuracy (0,76) en validation montre que l'accuracy est dominée par le fond (le réseau détecte des vrais négatifs pour les classes à classifier).
La perte est instable : pics aux époques 21 à 27 (jusqu'à 10 en entraînement), puis explosion aux époques 43 à 49 (jusqu'à 42,5 à l'époque 44, et 4,1 en validation), alors que F1 et accuracy continuent de monter. Quelques pixels sont prédits faux avec une confiance très élevée. La perte commence à redescendre à partir de l'époque 45 (de 42,5 à 12 en entraînement, de 4,1 à 0,44 en validation), juste après que `ReduceLROnPlateau` a probablement divisé le taux d'apprentissage par 10 (11 époques sans amélioration de la perte de validation depuis l'époque 34). C'est un indice que le taux de 1e-3 est trop élevé pour un encodeur pré-entraîné.
Le modèle sauvegardé est celui de l'époque 49, très proche du meilleur F1 (0,303 à l'époque 48).

6.2 Apport de nos augmentations

Même modèle et mêmes réglages, avec les augmentations de la § 2.3, puis sans aucune augmentation pour disposer d'une référence au même découpage (graine 42) :


U-Net ResNet-18, nos augmentations
Perte (Focal Loss)
entraînement
validation
époque
F1 macro
époque
Accuracy
époque


U-Net ResNet-18
augmentations d'origine (sans graine)
aucune augmentation (graine 42)
nos augmentations (graine 42)
niveaux de gris, non crédible (graine 42)
F1 macro (validation)
époque



Augmentations	F1 train	F1 valid (dernière époque)	F1 valid (moyenne des 5 dernières)	Accuracy valid	Perte valid (min)	Écart F1 train / valid
d'origine (§ 6.1, sans graine)	0,404	0,300	0,288	0,760	0,439 (ép. 49)	0,104
aucune (graine 42)	0,502	0,380	0,352	0,759	0,486 (ép. 43)	0,122
nos augmentations (graine 42)	0,473	0,408	0,342	0,765	0,461 (ép. 44)	0,065
niveaux de gris, non crédible (graine 42)	0,552	0,382	0,367	0,770	0,507 (ép. 47)	0,170

Sur 50 époques, nos augmentations n'améliorent pas nettement le F1 de validation par rapport à l'absence d'augmentation (même découpage) : 0,408 contre 0,380 à la dernière époque, mais 0,342 contre 0,352 en moyenne sur les 5 dernières. Le F1 oscille fortement d'une époque à l'autre en fin d'entraînement (entre 0,28 et 0,41 pour nos augmentations) : l'écart entre les deux runs reste dans ce bruit.

Sans augmentation, la courbe de validation est au-dessus pendant la plus grande partie de l'entraînement : les augmentations rendent la tâche plus difficile et ralentissent l'apprentissage. Le modèle n'ayant pas convergé, leur bénéfice pourrait n'apparaître qu'avec un entraînement plus long.

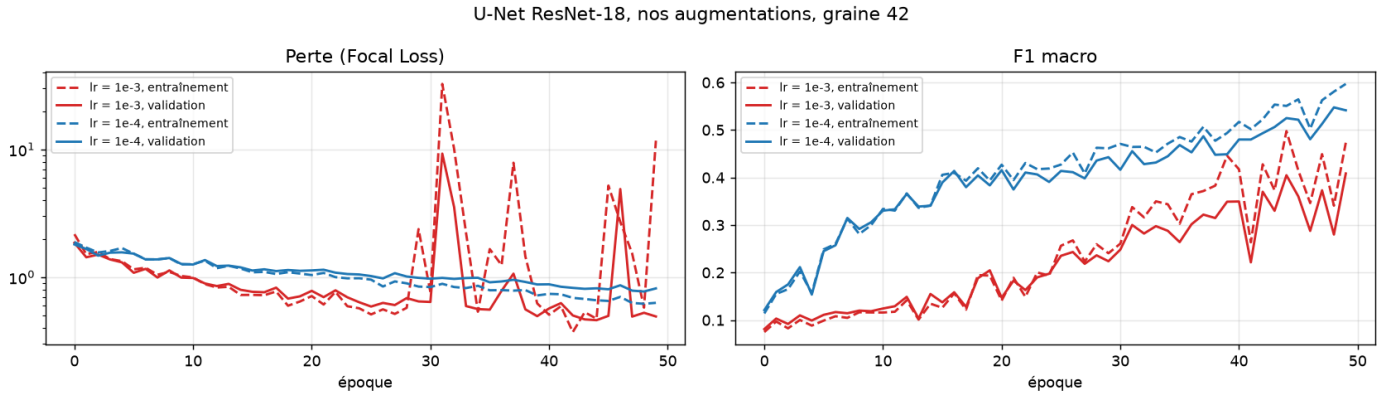
Nos augmentations réduisent en revanche nettement le sur-apprentissage : l'écart entre entraînement et validation passe de 0,122 à 0,065. Les métriques d'entraînement étant calculées sur les images augmentées, une partie de ce rapprochement vient aussi de la difficulté accrue des images d'entraînement.

Le gain apparent par rapport à la baseline (0,30) vient surtout de la suppression des augmentations d'origine : même sans aucune augmentation, le U-Net fait mieux (0,38).

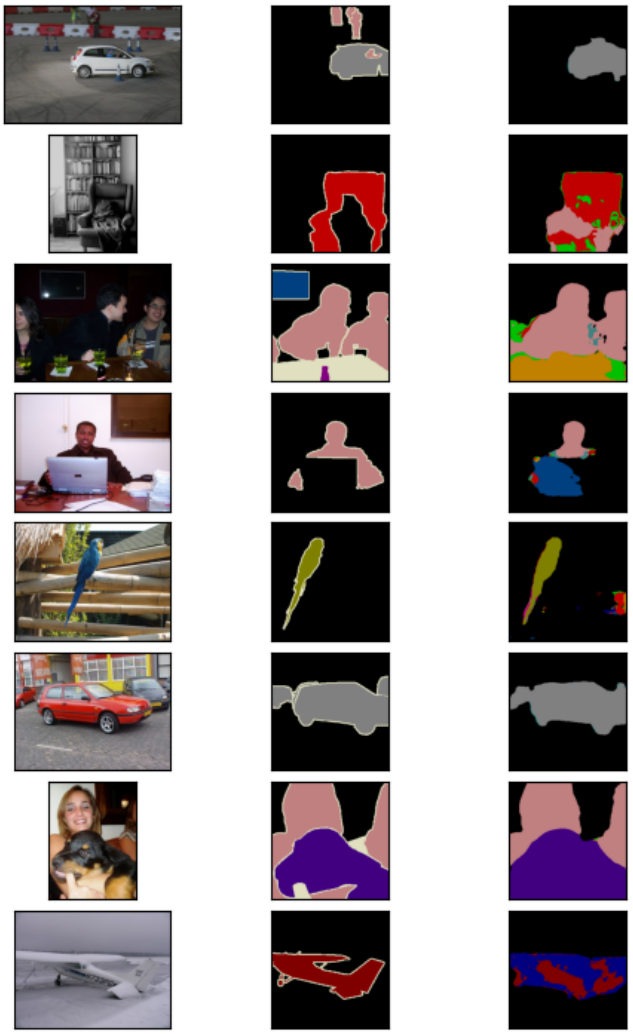
L'accuracy ne bouge presque pas (de 0,759 à 0,765) : seul le F1 macro, qui donne le même poids à chaque classe, met en évidence les différences sur les objets.
Augmentation volontairement non crédible : l'entraînement entièrement en niveaux de gris avec une validation en couleur, devait créer un décalage de distribution et dégrader le F1. Ce n'est pas le cas : 0,382 à la dernière époque et 0,367 en moyenne sur les 5 dernières, soit autant que sans augmentation, dans la marge de variation du F1. À ce stade d'entraînement, le U-Net s'appuie surtout sur les formes et les textures, que le passage en gris conserve, plus que sur la couleur. L'écart entre entraînement et validation est en revanche le plus grand des runs (0,170), signe que le modèle généralise moins bien vers les images en couleur.
Le modèle n'a pas convergé : le F1 de validation progresse encore à la fin.

6.3 Effet du taux d'apprentissage

Les pics de perte observés dans tous les runs du U-Net, et leur atténuation lorsque `ReduceLROnPlateau` divise le taux d'apprentissage, suggèrent qu'un taux de 1e-3 est trop élevé. Le U-Net avec nos augmentations a donc été réentraîné avec un taux dix fois plus faible, toutes choses égales par ailleurs :



Taux d'apprentissage	F1 train	F1 valid (dernière époque)	F1 valid (moyenne des 5 dernières)	Accuracy valid	Perte d'entraînement max	Écart F1 train / valid
1e-3	0,473	0,408	0,342	0,765	32,6	0,065
1e-4	0,597	0,541	0,521	0,808	1,9	0,056



Le taux d'apprentissage est de loin le réglage le plus influent : à 1e-4, le F1 de validation passe à 0,541 (0,521 en moyenne sur les 5 dernières époques), contre 0,342 à 1e-3. L'écart dépasse largement la variation d'un run à l'autre (environ 0,03).
L'apprentissage devient stable : la perte décroît régulièrement et le F1 progresse presque continûment au lieu d'osciller. Cela confirme que les instabilités observées jusqu'ici venaient d'un taux trop élevé : des pas trop grands dégradent les poids pré-entraînés de l'encodeur et font diverger certaines prédictions.
Le modèle apprend aussi plus vite : le F1 de validation dépasse 0,40 dès l'époque 16, un niveau que le run à 1e-3 n'atteint qu'à l'époque 44.
L'écart entre entraînement et validation reste faible (0,056) : le modèle ne sur-apprend pas, et le F1 progresse encore à la dernière époque. Un entraînement plus long devrait encore l'améliorer.
La perte de validation est pourtant plus élevée à 1e-4 (0,82 contre 0,49 à la dernière époque) alors que le F1 est bien meilleur : le modèle est moins « sûr de lui » (probabilités moins tranchées, donc perte plus forte) mais choisit plus souvent la bonne classe.

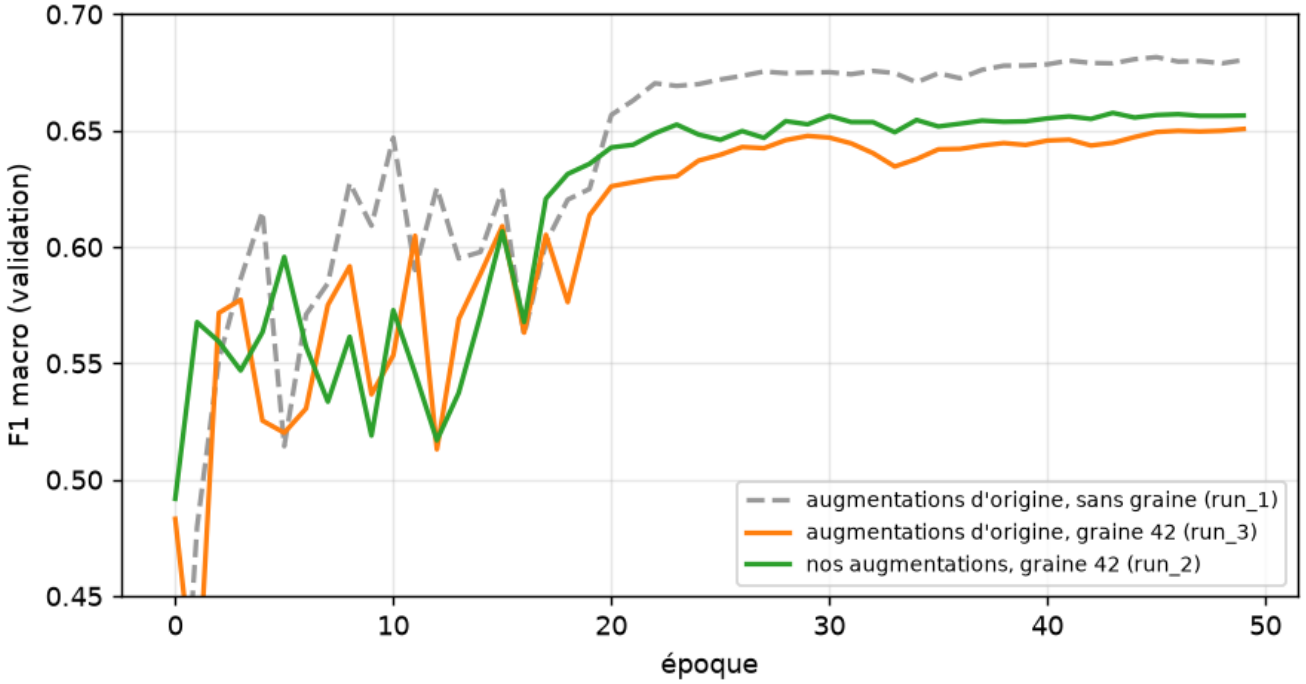
6.4 Comparaison U-Net / DeepLabV3

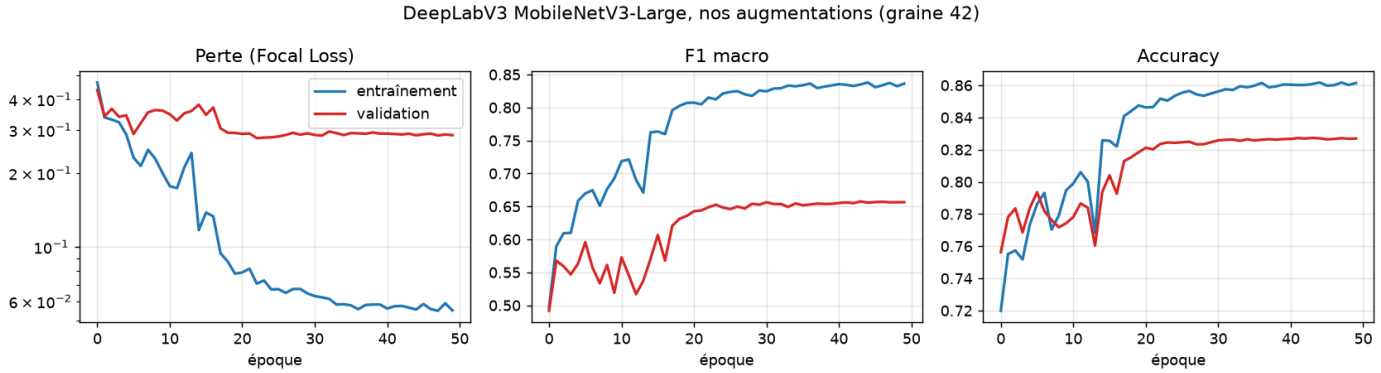
Trois runs de DeepLabV3 ont été réalisés, tous sur 50 époques :

Run	Augmentations	Graine
`run_1`	d'origine	aucune (ancien code)
`run_3`	d'origine	42
`run_2`	nos augmentations	42

Les runs avec la graine 42 (`run_2` , `run_3` et `unet_complete_1`) utilisent exactement les mêmes images de validation, ce que confirment les figures de prédiction enregistrées dans TensorBoard. Les runs sans graine (`run_0` , `run_1`) utilisent chacun un autre découpage.

Effet de nos augmentations sur DeepLabV3. À découpage identique, nos augmentations n'apportent pratiquement rien à DeepLabV3 : F1 de validation de 0,656 contre 0,651 avec les augmentations d'origine, soit +0,006, un écart négligeable.

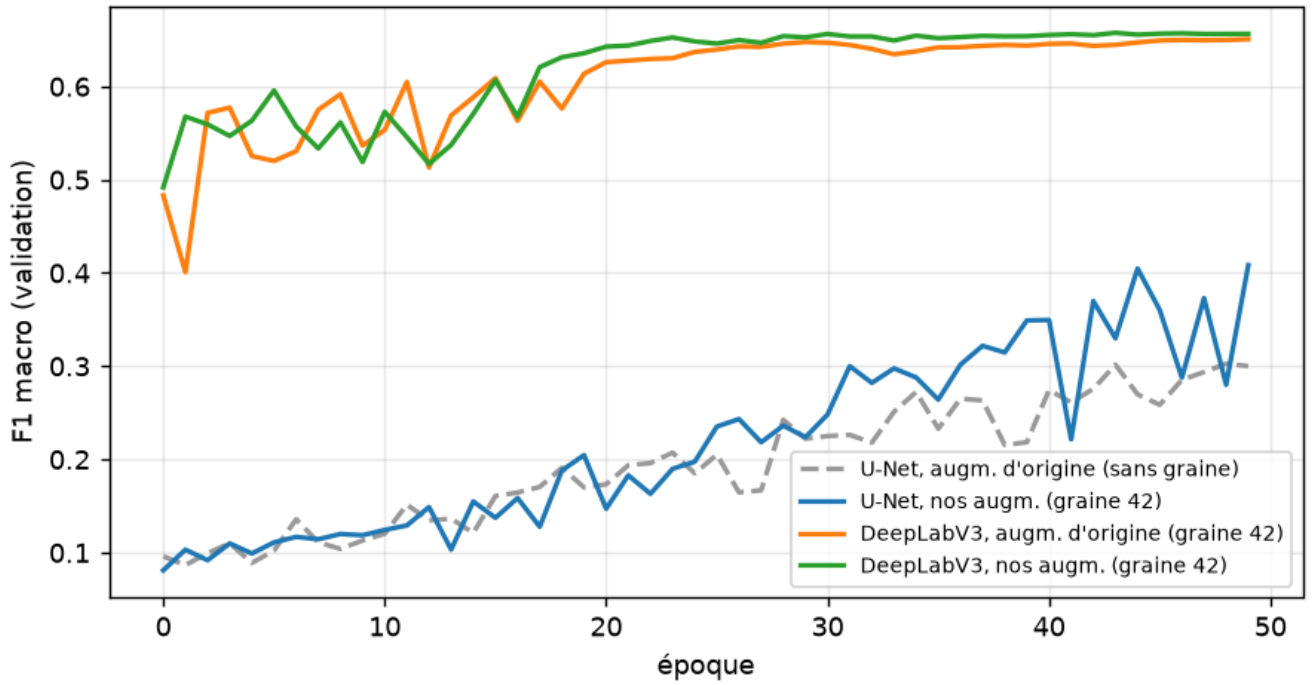


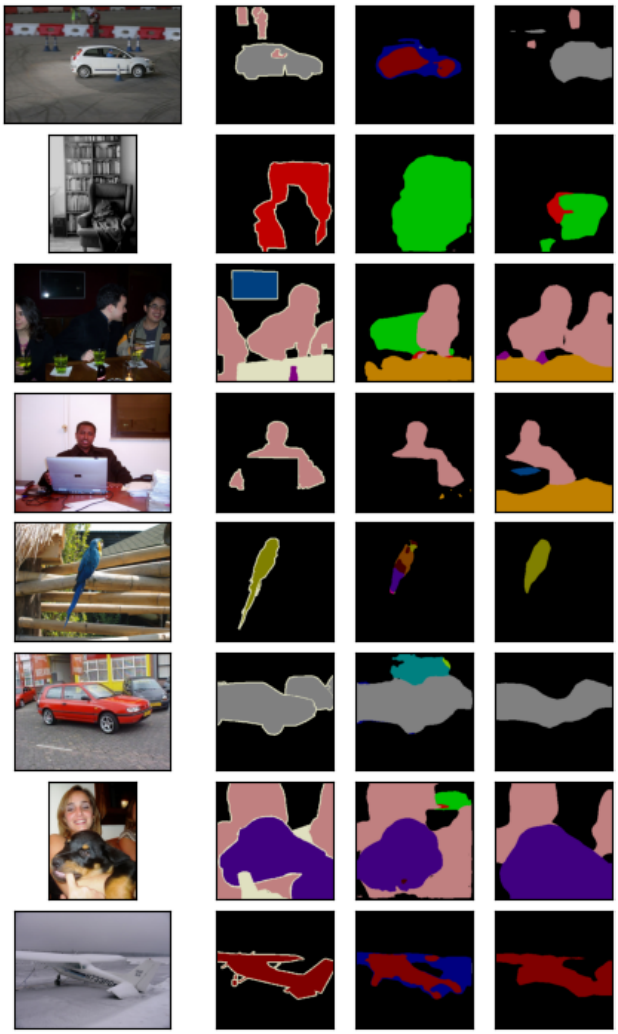


Modèle	Augmentations	Graine	Paramètres	F1 train	F1 valid (dernière)	F1 valid (moy. 5 dernières)	Accuracy valid	Perte valid (min)
U-Net ResNet-18	aucune	42	22,7 M	0,502	0,380	0,352	0,759	0,486 (ép. 43)
U-Net ResNet-18	nos augmentations	42	22,7 M	0,473	0,408	0,342	0,765	0,461 (ép. 44)
U-Net ResNet-18, lr = 1e-4	nos augmentations	42	22,7 M	0,597	0,541	0,521	0,808	0,773 (ép. 48)
DeepLabV3 MobileNetV3-Large	d'origine	42	11,0 M	0,859	0,651	0,650	0,826	0,277 (ép. 32)

Modèle	Augmentations	Graine	Paramètres	F1 train	F1 valid (dernière)	F1 valid (moy. 5 dernières)	Accuracy valid	Perte valid (min)
DeepLabV3 MobileNetV3-Large	nos augmentations	42	11,0 M	0,836	0,656	0,657	0,827	0,278 (ép. 22)

Seuls les runs avec la graine 42 figurent dans ce tableau : ils partagent le même découpage train / valid et sont directement comparables. Durées d'entraînement : environ 17 min pour le U-Net et 9,7 min pour DeepLabV3.

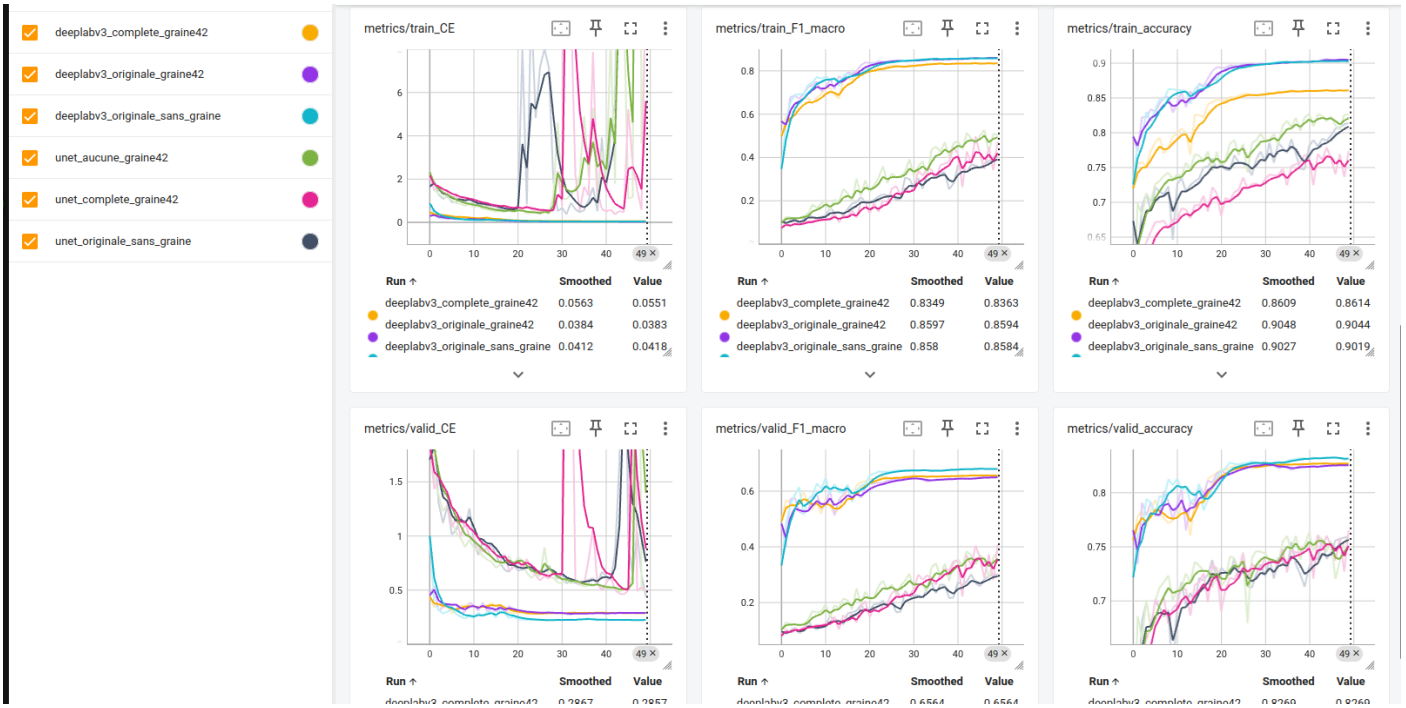




Les deux dernières colonnes comparent les prédictions des deux modèles sur les mêmes images, à la dernière époque.

DeepLabV3 reste devant le U-Net (F1 de validation de 0,656 contre 0,541 pour le meilleur U-Net, entraîné avec un taux d'apprentissage de 1e-4, à découpage identique). L'écart est passé de 0,25 à 0,12 grâce au seul taux d'apprentissage. DeepLabV3 y parvient avec deux fois moins de paramètres (11,0 M contre 22,7 M).
Pour le U-Net, seul l'encodeur est pré-entraîné, sur de la classification ImageNet, et le décodeur part de zéro avec 1 171 images, contrairement à DeepLabV3, entièrement pré-entraîné (encodeur et tête de segmentation) à segmenter les classes de VOC.
Sur 50 époques, nos augmentations n'améliorent le F1 de validation d'aucun des deux modèles de façon significative. Elles réduisent en revanche le sur-apprentissage du U-Net, dont le décodeur part de zéro.
DeepLabV3 converge : F1 et perte se stabilisent à partir de l'époque 20, probablement quand `ReduceLROnPlateau` divise le taux d'apprentissage par 10. Le F1 de validation plafonne ensuite vers 0,65. L'écart entre entraînement et validation reste important (F1 de 0,84 contre 0,66) : le modèle sur-apprend, et nos augmentations ne le réduisent que légèrement (0,18 contre 0,21).
Sur les mêmes images, DeepLabV3 donne plus souvent la bonne classe (voiture, perroquet, avion) avec des contours plus nets. Le U-Net retrouve la forme des objets mais confond les classes (voiture en partie prédite comme bus, avion comme bateau).

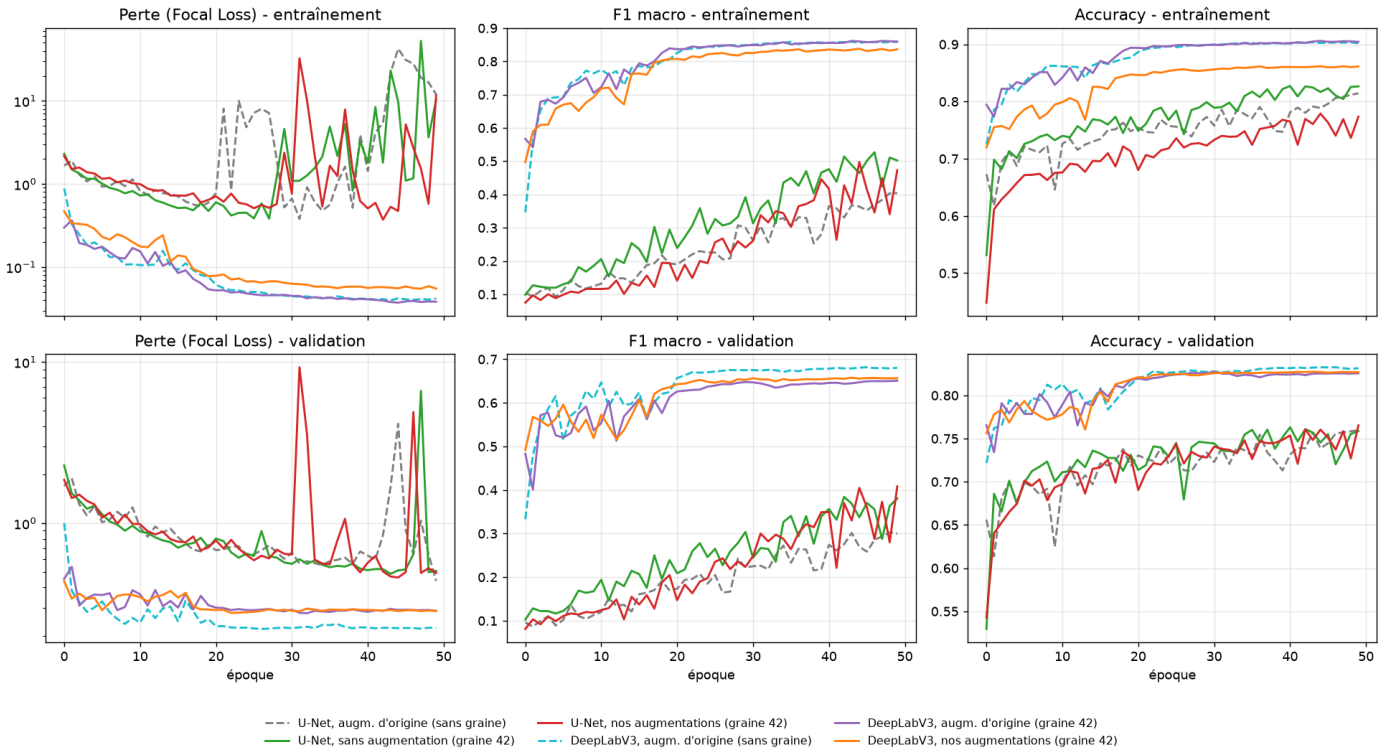
6.5 Vue d'ensemble des entraînements



Cette capture superpose dans TensorBoard les six runs réalisés, identifiés par couleur dans la colonne de gauche :

Couleur	Run	Modèle	Augmentations	Graine
orange	`deeplabv3_complete_graine42`	DeepLabV3	nos augmentations	42
violet	`deeplabv3_originale_graine42`	DeepLabV3	d'origine	42
bleu clair	`deeplabv3_originale_sans_graine`	DeepLabV3	d'origine	aucune
vert	`unet_aucune_graine42`	U-Net ResNet-18	aucune	42
rose	`unet_complete_graine42`	U-Net ResNet-18	nos augmentations	42
gris foncé	`unet_originale_sans_graine`	U-Net ResNet-18	d'origine	aucune

La ligne du haut donne les métriques sur le pli d'entraînement, celle du bas sur le pli de validation ; de gauche à droite : la perte, le F1 macro et l'accuracy, en fonction de l'époque. La courbe foncée de chaque couleur est lissée par TensorBoard, la courbe claire donne les valeurs brutes.



Cette seconde figure reprend exactement les mêmes données, tracées directement à partir des fichiers de logs : toutes les courbes et la légende sont lisibles en même temps, et la perte est affichée en échelle logarithmique pour que ses explosions ne masquent pas le reste. Les traits pleins correspondent aux runs avec la graine 42, comparables entre eux ; les tirets aux deux premiers runs, sans graine.

Les deux familles de modèles se séparent nettement : les trois DeepLabV3 atteignent environ 0,6 de F1 de validation en une dizaine d'époques, les trois U-Net de cette vue restent sous 0,4 après 50 époques.
Les DeepLabV3 convergent (courbes plates après l'époque 20) avec un écart marqué entre entraînement et validation ; les U-Net progressent encore à la dernière époque et n'ont pas convergé.
Seules les pertes des U-Net présentent de fortes explosions, celles des DeepLabV3 restent stables.
Au sein de chaque famille, les écarts liés aux augmentations sont faibles devant l'écart entre familles.

7. Conclusion et perspectives

Un U-Net à encodeur ResNet-18 pré-entraîné atteint un F1 macro de validation de 0,30 avec les augmentations d'origine du code, et d'environ 0,35 à 0,40 sans augmentation ou avec nos augmentations. Sur 50 époques, nos augmentations n'améliorent pas significativement le F1 par rapport à l'absence d'augmentation, mais elles réduisent nettement le sur-apprentissage ; leur bénéfice pourrait apparaître avec un entraînement plus long. Le réglage le plus influent s'est révélé être le taux d'apprentissage : à 1e-4 au lieu de 1e-3, l'entraînement devient stable et le F1 de validation du U-Net passe de 0,34 à 0,52 (moyenne des 5 dernières époques), sans sur-apprentissage. Le U-Net reste derrière DeepLabV3 (0,66), dont les poids sont déjà entraînés à segmenter les classes de VOC, mais l'écart est divisé par deux : le pré-entraînement sur la tâche explique l'essentiel de la différence restante, plus que l'architecture ou les augmentations, qui n'apportent d'ailleurs rien à DeepLabV3.

Les expériences ont aussi montré l'importance du protocole. Sans graine fixe, le découpage train / valid change d'un run à l'autre et suffit à faire varier le F1 de 0,03 ; il faut donc le fixer avant de comparer des réglages, et comparer de préférence la moyenne des dernières époques, le F1 du U-Net oscillant fortement d'une époque à l'autre. Enfin, la perte de validation seule est un mauvais critère de comparaison : le U-Net à 1e-4 a une perte plus élevée qu'à 1e-3 mais un F1 bien meilleur.

Perspectives :

- entraîner plus longtemps avec un taux d'apprentissage de 1e-4, le U-Net n'ayant pas convergé en 50 époques ;
- utiliser un taux d'apprentissage plus faible pour l'encodeur pré-entraîné que pour le décodeur ;
- comparer les architectures à pré-entraînement égal, en chargeant DeepLabV3 avec seulement un encodeur pré-entraîné sur ImageNet ;

- tester d'autres encodeurs avec le `GenericUNet` (EfficientNet), la bibliothèque `segmentation_models_pytorch` et DeepLabV3+ ;
- évaluer avec le mIoU par classe, la métrique de référence sur VOC.